

Lab report # 12

1. Objective

The objective of this lab is to:

- Understand computer memory and variables
 - Learn pointers in C++
 - Understand pointer declaration
 - Use referencing operator &
 - Use dereferencing operator *
-

2. Theory

2.1 Computer Memory

Computer memory is made up of bytes. Each byte has a unique address. These addresses are used by the system to store and access data.

A variable is stored in a specific memory location, and that location is accessed using its address.

2.2 Variable and Memory Relationship

A variable is a named memory location that stores data. When a variable is declared, the compiler reserves memory for it.

Example:

```
#include <iostream>
using namespace std;

int main()
{
    float f1 = 3.14; // yahan float variable ko memory allocate ki ja rahi
    hai

    cout << f1; // yahan variable ki value display ki ja rahi hai

    return 0;
}
```

Explanation:

- The variable `fl` is stored in memory.
 - The value is accessed from its memory address.
-

3. Pointers in C++

A pointer is a variable that stores the memory address of another variable.

- Pointer value is an address, not a direct value
 - It “points to” another variable
 - It has two parts: address and value at that address
-

4. Pointer Declaration

Syntax:

```
dataType *pointerName;
```

Example:

```
int *ptr;  
int k;
```

Explanation:

- `ptr` is a pointer variable
 - `*` shows that it is a pointer
 - `int` shows it stores address of an integer
-

5. Referencing Operator (&)

The referencing operator is used to get the address of a variable.

Example:

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int k = 10; // yahan variable ko value assign ki ja rahi hai
```

```

    int *ptr; // yahan pointer declare kiya ja raha hai

    ptr = &k; // yahan k ka address pointer ko diya ja raha hai

    cout << "Value of k: " << k << endl; // yahan k ki value display ho rahi
    hai
    cout << "Value using pointer: " << *ptr << endl; // yahan pointer ke
    through value access ki ja rahi hai

    return 0;
}

```

6. Dereferencing Operator (*)

The dereferencing operator is used to access the value stored at the address.

Example:

```

#include <iostream>
using namespace std;

int main()
{
    int k = 5; // yahan variable initialize kiya gaya hai

    int *ptr = &k; // yahan pointer ko address assign kiya gaya hai

    *ptr = 7; // yahan pointer ke through value change ki ja rahi hai

    cout << "Value of k: " << k << endl; // yahan updated value display ho
    rahi hai

    return 0;
}

```

7. Pointer Manipulation Example

```

#include <iostream>
using namespace std;

int main()
{
    int a = 5; // yahan pehla variable initialize kiya gaya hai
    int b = 15; // yahan doosra variable initialize kiya gaya hai

    int *p1, *p2; // yahan do pointer declare kiye gaye hain

    p1 = &a; // yahan p1 ko a ka address diya gaya hai
    p2 = &b; // yahan p2 ko b ka address diya gaya hai
}

```

```
*p1 = 10; // yahan a ki value change ki ja rahi hai
*p2 = *p1; // yahan b ko p1 ki value di ja rahi hai

*p1 = 20; // yahan dobara a ki value change ki ja rahi hai

cout << "a = " << a << endl; // yahan a ki final value display ho rahi
hai
cout << "b = " << b << endl; // yahan b ki final value display ho rahi
hai

return 0;
}
```

8. Output

```
a = 20
b = 10
```

9. Conclusion

In this lab, we learned:

- Computer memory and variable storage
- Pointer concept and its working
- Address operator & usage
- Dereference operator * usage
- Pointer-based value manipulation in memory